

PORYGON — Learning What “Normal” Looks Like Inside a Running Container

Does knowing how a container was deployed make its behaviour easier to judge?

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: [XX]

S.No	Register Number	Student Name
1	CB.EN.U4CSE23XXX	Anurup R Krishnan
2	CB.EN.U4CSE23XXX	Student Name 2
3	CB.EN.U4CSE23XXX	Student Name 3
4	CB.EN.U4CSE23XXX	Student Name 4

Guide: **Guide Name**, Designation, Department of CSE

September 5, 2026

The Problem, in Plain Words

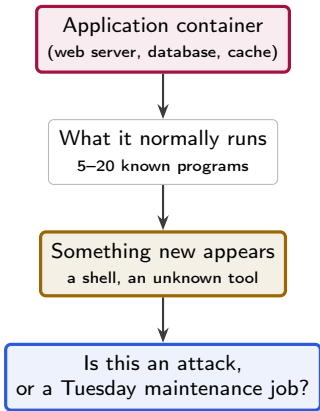
Why watching a container is harder than it sounds

A **container** is a packaged application that runs isolated on a server. Companies run thousands of them.

If an attacker gets inside one, they run **programs** that the application would never normally run — a command shell, a download tool, a scanner.

So the natural idea is: *learn what programs a container normally runs, then flag the unusual ones.*

The catch. To compare today against “normal”, you must first decide *which normal*. Group the wrong containers together and the picture blurs.



The honest answer is often “we cannot tell yet”.
Our design keeps that uncertainty visible instead of
hiding it.

What We Are Actually Asking

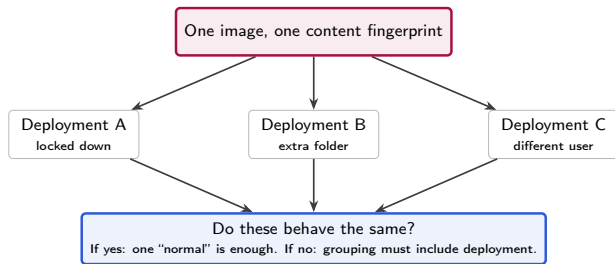
The research question, and why it is not obvious

Every container image has a **content fingerprint** — a long code computed from the exact bytes of the image. Same code, same program. It cannot be faked or quietly changed.

So: group containers by fingerprint, learn one “normal” per fingerprint. Done?

Not quite. The same image can be *deployed* in different ways — different start-up command, different user account, different permissions, different folders attached. Those differences may change behaviour.

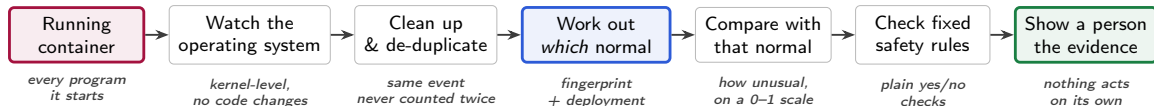
Our question: does adding *how it was deployed* to the grouping give a cleaner “normal” — or just split the data into pieces too small to learn from?



We compare **four** ways of grouping and let the measurements decide. A “no difference” answer is a valid result, not a failure.

1. Methodology — How the System Works, End to End

Rubric: Workflow & Algorithmic Pipeline (5 Marks)



Two kinds of evidence, kept apart on purpose

- **Statistical:** “this is rare compared with known-normal runs”
- **Rule-based:** “a command shell started in a web server”

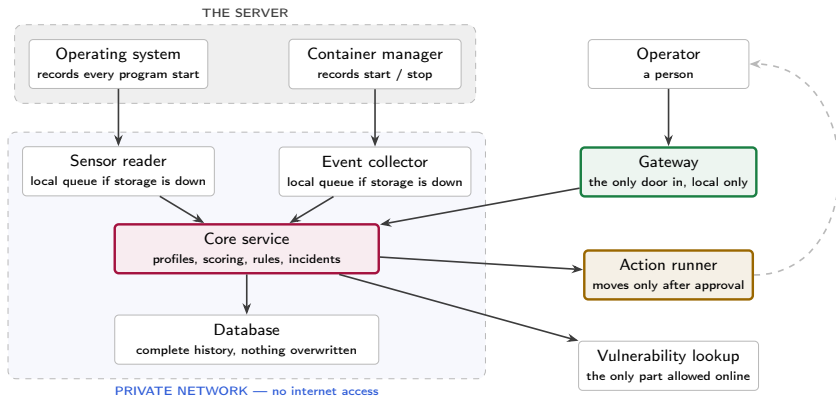
Mixing them into one number hides which one fired.

What is deliberately *not* in the loop

- No automatic shutting down of containers
- No machine-learning black box
- No claim that “unusual” means “attack”

1. Methodology — System Architecture

Rubric: Architecture & Trust Boundaries (5 Marks)



Each part gets the **least** access it needs. The core service never reaches the internet; the vulnerability lookup never touches the database; the action runner only moves when a person has approved.

1. Methodology — The Mathematics

Rubric: Mathematical Model / Formulation (5 Marks)

Step 1 — How far has the mix of programs shifted?

Compare the proportions seen now (q) with normal (p):

$$H(p, q) = \sqrt{\frac{1}{2} \sum_i (\sqrt{p_i} - \sqrt{q_i})^2}$$

Always between 0 and 1. Zero means identical.

Step 2 — How surprising is the order?

Programs start in habitual sequences. Score each step by how unexpected it is:

$$S = -\frac{1}{n} \sum \log \hat{P}(\text{next} \mid \text{current})$$

Unseen pairs get a small non-zero chance, so nothing divides by zero.

Step 3 — How much is brand new?

The share of activity whose program was *never* seen while learning.

Step 4 — Combine, without invented weights

Each measure is turned into a rank against known-normal runs, then **averaged equally**. If a measure is missing, it is recorded as missing — not guessed.

Step 5 — Turn it into a fair scale

Take n complete known-normal runs held back and never used for learning:

$$p = \frac{1 + \#\{\text{normal runs at least this odd}\}}{n + 1}$$

Read it as: “about 1 in 20 normal runs look this unusual” — *not* “95% chance of attack”.

Why no tuned weights? An earlier version mixed the measures using 0.50, 0.30, 0.20. Nobody could justify 0.50 over 0.45. Tuning those numbers to make results look good is exactly the mistake we are avoiding, so the code now refuses to accept weights at all.

1. Methodology — Data, Splits and Evaluation Metrics

Rubric: Datasets, Preprocessing & Evaluation Metrics (5 Marks)

The data is generated, not downloaded.

No public dataset matches this question, so we produce our own under controlled conditions: three real, widely used server programs, each locked to an exact image fingerprint.

Program	Locked version
Web server	nginx 1.26.3
In-memory store	Redis 7.2.7
Database	PostgreSQL 16.6

Each is run idle, under steady load, under bursts, and with deployment changes. Unusual-but-harmless jobs (backups, restarts, maintenance) are recorded too — those are the cases that cause false alarms.

Splitting rule: a *whole run* goes to learning, calibration, or testing — never split inside a run. Snapshots seconds apart are near-copies; treating them as independent would make our confidence

What we will measure

Group	Measures
Getting it right	false-alarm rate on harmless runs; how many planted events are caught
Trustworthy data	events created vs. events stored, at every hand-off point
Speed	delay from event to stored record (typical, slow, worst)
Cost	processor time, memory, disk growth, slowdown of the watched app
Honesty of scale	do 95% of normal runs really score inside the 95% band?

Rule we hold ourselves to: every percentage is reported with the actual counts behind it. No number appears without the runs that produced it.

2. Implementation Progress — Module Status

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

Module	What it does today	Status
Kernel-level watcher	Every program start, in every container	Working
Container event collector	Start/stop events; resolves the exact image identity	Working
Safe delivery	Local queue; never skips an event it failed to store	Working
Storage & history	9 schema upgrades; nothing overwritten	Working
Deployment fingerprint	Turns a deployment into one repeatable identity	Working
Behaviour profiles	Builds and versions “normal”; refuses weak ones	Working
Unusualness scoring	Explainable score with its reasons attached	Working
Rule-based evidence	Fixed checks, incident timeline, human sign-off	Working
Approved action runner	Pause or stop, only after a person approves	Working
Vulnerability lookup	Ingredient list per image; known-exploited flags	Working
Experiment harness	Runs real containers; tamper-evident results	Working
Four-way comparison	The core research comparison	Not started
Result tables, figures	Cannot exist before the study runs	Not started

Overall $\approx 65\%$ — platform $\approx 90\%$ complete, evaluation $\approx 15\%$. We report the split, not one flattering number.

2. Implementation Progress — Evidence It Actually Runs

Rubric: Functional & Integrated (8 Marks)

Size and coverage

Lines of program code	17,802
Automated tests passing	156
Service interfaces	64
Database upgrade steps	9
Services running together	8

Automatic checks, all passing

Code & configuration	pass
All unit tests	pass
Live end-to-end run	pass
Real-container run	pass

Measured on real containers (16 runs)

Marked test events created	96
Seen by the kernel watcher	96
Stored in the database	96
Lost	0
Counted twice	0
Application requests served	360/360
Leftover containers after cleanup	0

Complete chain proven on real data



From a real database container: 479 recorded program starts became a profile, a later window scored *normal-looking*, and 72 rule checks matched without raising a false incident.

2. Implementation Progress — The Safety Rails Work

Live demonstration: the refusals are the interesting part

A system like this is only trustworthy if it says **no** at the right moments. Three refusals we can show live:

1. It refuses a weak profile.

Asked to learn “normal” from too little data, it stops:

`needed 3 time windows, found 2`

2. It refuses to mark its own homework.

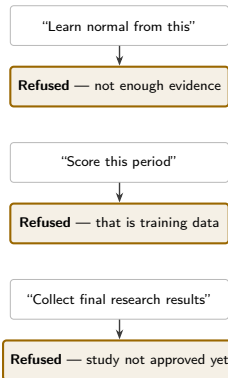
Asked to score a period it already learned from:

`scoring windows must not overlap the training interval`

3. It refuses a moving target.

Asked to use a version label instead of an exact fingerprint:

`refusing a mutable image reference`



Each refusal is enforced by code and covered by a test, so it cannot be bypassed by accident.

3. Technical Knowledge — Key Decisions and Why

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Decision	Why	What we rejected
Identify images by content fingerprint	A version label can be pointed at different software tomorrow; a fingerprint cannot	Grouping by container name or version label
Watch at the operating-system level	Sees every program start without changing the application	Modifying applications, or trusting logs the attacker can edit
Use a maintained sensor	Same evidence, far less risk than writing our own kernel code	Building a custom kernel sensor — large effort, large attack surface
Combine measures by equal average	No number we cannot justify to an examiner	Weights tuned until the results look good
Judge against held-back normal runs	Turns a raw distance into “how often does normal look this odd?”	Declaring a threshold and calling it accuracy
Split by whole runs	Snapshots seconds apart are near-copies; reusing them fakes confidence	Random splitting of time windows
A person approves every action	A false alarm must never take down a healthy service	Automatic shutdown on a score

3. Technical Knowledge — Trade-offs We Accepted

Every choice costs something; here is the bill

We gained	We paid
Strong, cheap signal from program starts	Blind to file access and network destinations
Cleaner “normal” per deployment	Fewer runs per group; some groups too small to judge
No unjustifiable tuned numbers	Almost certainly not the most accurate possible
Honest confidence intervals	Need far more runs for the same certainty
A fair, interpretable scale	Only valid while the workload does not drift
No self-inflicted outages	Slower response to a real attack
One language across all services	A known write-speed ceiling, not yet a bottleneck

Edge cases already handled

- Storage full → queue locally, never skip an event
- Sensor restarts → counters do not go backwards
- Same event twice → stored once
- Malformed input → kept as a short, scrubbed sample
- Too little data → answer “insufficient”, never guess
- Secret on a command line → replaced before storage

3. Technical Knowledge — Early Observations

Rubric: Preliminary Observations (5 Marks) — measured, not expected

1. The pipeline loses nothing (so far).

96 marked events created, 96 seen by the kernel watcher, 96 stored. Points we *cannot* yet measure are labelled “not measured” — never assumed to be zero.

2. A bug that only two runs could reveal.

The deployment identity accidentally included a name that changes every run, so the same deployment looked different each time. Left unfixed, every group would have stayed permanently “too small to judge” — and it would have looked like a finding rather than a defect.

3. Serving traffic starts no programs.

40 web requests and 40 database-cache operations produced *zero* recorded program starts. What we profile is start-up and maintenance activity — worth stating plainly rather than overselling.

4. The deployment changes we tested made no behavioural difference.

Program	Variants	Events each	Same?
Web server	3	46 / 46 / 46	yes
Database	3	166 / 166 / 166	yes
Cache	3	33 / 33 / 33	yes

Restricting permissions and adding a scratch folder changed the *identity* but not one program that ran.

Why this matters now: running the full study with these settings would have produced a “no benefit” answer caused by our own choice of test conditions — not by the idea being wrong. We will switch to deployment changes that actually alter which programs run, *before* the study is locked.

Catching this early is exactly what a trial run is for.

3. Technical Knowledge — What We Do Not Claim

Knowing the limits is part of understanding the method

Unusual is not the same as malicious.

- Unusual but harmless: backups, debugging, restarts, a traffic spike
- Harmful but ordinary-looking: an attacker using tools already present

We keep the two ideas in separate fields and never merge them into one verdict.

What the system can miss

- Anything that is not a program start
- Slow activity spread thinly over time
- Behaviour that was already present while learning
- Any container whose group has too few examples

Not yet claimed, and why

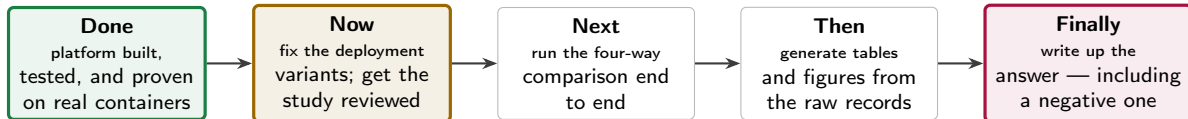
Claim	Blocked by
Detection accuracy	final study not run
False-alarm rate	final study not run
Which grouping wins	the comparison itself
Suitable for production	single machine, research scope

Current blockers

- 1 Study design awaiting two independent human reviews — the code refuses to collect final results until then
- 2 65 GB free disk against 150 GB needed for the full study
- 3 Deployment variants need revising (previous slide)

Where This Goes Next

From a working platform to an answered question



The contribution we are aiming for is not “we watched containers” — that is well-established work. It is a careful, repeatable comparison of *four different ways of deciding what counts as the same thing*, reported honestly whichever way it comes out.

Every number will be traceable to the exact run that produced it, and every run is stored so a reviewer can recompute the tables themselves.

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- Individual MOOC / Online Certification Selection:

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Anurup R Krishnan	[e.g. Container Security / Linux Observability]	Coursera	15 Hrs
2	Student Name 2	[e.g. Applied Statistics for Experiments]	NPTEL	20 Hrs
3	Student Name 3	[e.g. Docker & Cloud-Native Fundamentals]	edX / Coursera	12 Hrs
4	Student Name 4	[e.g. Cloud Security Foundations]	AWS / Google	16 Hrs

- Requirement:** Each student must complete one approved 10–20 hour online certification from a recognized platform (Coursera, NPTEL, edX, etc.) relevant to the project.
- Note:** course titles above are suggestions aligned to the project's four skill areas — container internals, operating-system observation, experiment design, and security practice. Replace with the actual enrolled courses and attach completion certificates.



S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symp. Security and Privacy*, 1996, pp. 120–128.



S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.



A. S. Abed, T. C. Clancy, and D. S. Levy, "Applying bag of system calls for anomalous behavior detection of applications in Linux containers," in *Proc. IEEE Globecom Workshops*, 2015, pp. 1–5.



J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.



V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.



G. Shafer and V. Vovk, "A tutorial on conformal prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.



B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Boston, MA, USA: Addison-Wesley, 2019.



The Falco Authors, "Falco: cloud-native runtime security," Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, "OCI image format specification." [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You

Questions welcome — including the uncomfortable ones about what this system cannot yet prove.